5. AI, LLMs and Agentic Workflows – Interview Notes

Short brush-up notes. Sections A and B are general concepts. Section C is a framework for explaining your own projects.

Important for Section C: I don't have the actual details of your projects. The answers there are a template based on how such systems are typically built. Replace anything in [square brackets] with what you really implemented, and delete anything you did not do. Interviewers quickly spot answers that don't match the real code.

A. LLM Fundamentals

1. What is an LLM?

A Large Language Model (LLM) is a neural network trained on huge amounts of text to predict the next token. From that one skill it can summarize, answer questions, write code and SQL, and follow instructions.

Key points:

- It generates text based on patterns it learned. It does not "look up" facts unless you give it tools or data.
- Its knowledge has a cutoff date.
- It is probabilistic: the same prompt can give slightly different answers.

Interview answer:

"An LLM is a transformer-based model trained to predict the next token. It's good at understanding and generating language and code, but it can be wrong, so in production I pair it with tools, validation and grounded data."

2. Tokens and Context Windows

Token: a piece of text (a word, part of a word, or symbol). Roughly 1 token is about 4 characters or three quarters of an English word.

Context window: the maximum number of tokens the model can handle in one request. It includes everything: system prompt, history, retrieved documents, your question and the answer.

Why it matters:

- Cost and latency grow with the number of tokens.
- If the input is too long, it must be trimmed or summarized.
- Very long contexts can still reduce accuracy (the model may miss details in the middle).

3. Temperature and Model Output

Temperature controls randomness.

Temperature	Behaviour	Use for
Low (0 to 0.2)	More deterministic and consistent	SQL generation, extraction, JSON, validation
Medium (0.5 to 0.7)	Balanced	General answers

Temperature	Behaviour	Use for
High (0.8 and above)	More creative and varied	Brainstorming, creative writing

Note: even at 0 the output is not guaranteed to be identical every time. Other settings: max tokens (limits answer length) and top-p (limits the pool of likely tokens).

Remember: for automation and data work, use low temperature.

4. System Prompts vs User Prompts

System prompt	User prompt
Sets the role, rules and behaviour of the model	The actual task or question
Written by the developer	Provided at runtime (often by the user)
Example: "You are a BigQuery SQL assistant. Only generate SELECT queries."	Example: "Show duplicate customer IDs in dim_customer."
Higher priority for rules and format	Specific request

Good practice: put rules, output format and safety constraints in the system prompt. Never rely on the system prompt alone for security, because users can try prompt injection. Enforce critical rules in code.

5. Prompt Engineering Techniques

Technique	What it is	Example
Clear instructions	Say exactly what, how and in what format	"Return only JSON with keys risk and reason."
Role prompting	Give the model a role	"You are a senior data engineer."
Few-shot	Provide examples of input and output	Show 2 sample SQL questions with correct SQL
Chain of thought	Ask it to reason step by step	"Think step by step, then give the final answer."
Delimiters	Separate instructions from data	Use triple quotes or XML-like tags around input
Constraints	Define what not to do	"Do not invent table names."
Grounding	Supply the facts it should use	Paste the table schema into the prompt

Tip: be specific, give context (for example the schema), define the output format, and iterate by testing.

6. Structured JSON Output

For automation, you need predictable output that code can parse, not free text.

How to get it:

1. Ask for JSON in the prompt and give the exact schema or an example.
2. Use the model's structured output or JSON mode, or tool calling with a schema, where available.
3. Parse and validate in code (Pydantic or JSON Schema): check required keys, types and allowed values.
4. If validation fails, retry once with the error message, then fall back or fail safely.

Example schema:

```
{
  "risk_level": "LOW | MEDIUM | HIGH",
  "reasons": ["string"],
  "needs_human_review": true
}
```

Remember: never trust LLM output directly. Always validate before using it.

7. Hallucinations

A hallucination is a confident but incorrect or made-up answer (for example a table name or column that does not exist).

Why it happens: the model predicts likely text, not verified truth.

How to reduce it:

- Give the real data or schema in the prompt (grounding / RAG).
- Use low temperature.
- Constrain the output (JSON schema, allowed values).
- Validate the output with code (for example a SQL dry run to check tables and columns).
- Ask for sources or evidence, and verify them.
- Tell the model it may say "I don't know".
- Use human review for high-risk actions.

8. Context Management

Because the context window is limited, decide what goes in.

Techniques:

- Send only relevant information (for example only the needed table schemas, not the whole catalog).
- Summarize older conversation history.
- Use retrieval (RAG) to fetch relevant documents on demand.
- Truncate long logs, keeping the errors and the most recent lines.
- Keep stable instructions at the start and the current task at the end.
- Store state outside the model (in a database or a variable) instead of repeating it in every prompt.

Interview answer:

"I treat the context window as a budget. I include only the schema, rules and facts needed for the current step, summarize or drop old history, and keep state in my own code so each LLM call is small, focused and cheaper."

B. Agentic AI

1. What is an AI Agent?

An AI agent is a system where an LLM decides what to do next, uses tools to take actions, observes the results, and repeats until the goal is complete.

```
Goal -> LLM thinks -> picks a tool -> tool runs -> result back to LLM -> repeat -> final answer
```

Simple example: "Check if release R123 is safe." The agent calls the GitHub tool, then the Bamboo tool, then calculates a score, then writes a summary.

2. AI Agent vs Traditional Automation

Traditional automation (script, workflow)	AI agent
Fixed steps decided by the developer	The model decides the steps at runtime
Deterministic and predictable	Flexible but less predictable
Fails on unexpected input	Can adapt to new situations
Cheap and fast	Costs tokens and is slower
Best for well-defined, repeatable tasks	Best for open-ended or messy tasks

Key interview point: use the simplest solution that works. Use normal code for the deterministic parts (API calls, validation, calculations) and use the LLM only where language understanding or flexibility is needed. Many good systems are workflows with LLM steps, not fully autonomous agents.

3. Tool Calling / Function Calling

The LLM cannot call APIs by itself. You describe tools (name, purpose, parameters) to the model. The model replies with a structured request to call a tool. Your code runs it and sends the result back.

Example tool definition:

```
{
  "name": "get_build_status",
  "description": "Get the status of a Bamboo build",
  "parameters": { "plan_key": "string", "build_number": "integer" }
}
```

Flow:

1. User asks: "Did build 45 of plan ABC pass?"
2. Model returns: call get_build_status(plan_key="ABC", build_number=45).
3. Your code calls the real API and returns the result.
4. Model uses the result to answer.

Important: your code validates the parameters and decides whether the call is allowed. The model only proposes.

4. Agent Execution Cycle

```
1. Receive the goal
2. Plan / decide the next action
3. Call a tool
4. Observe the result
5. Update state / memory
6. Done? If no, go back to step 2. If yes, return the final answer
```

Always add limits: a maximum number of steps, timeouts, and a budget on tokens, to avoid infinite loops and runaway cost.

5. Multi-step Workflows

Complex tasks are broken into steps. Common patterns:

- Sequential: step 1 then step 2 then step 3 (fetch data, validate, summarize).
- Branching: different path based on a result (if tests failed, stop; if passed, continue).
- Parallel: independent steps at the same time (check GitHub and Bamboo together).
- Planner and executor: one step creates a plan, others execute it.
- Retry and fallback: handle failures at each step.

Good design: each step has a clear input, output and validation, and the state is stored so a failed run can be restarted from a safe point (similar to ETL restartability).

6. Agent Memory

Type	Meaning	Example
Short-term	Current conversation or task state, held in the context	Results of earlier tool calls in this run
Long-term	Stored across runs, in a database or vector store	Past incidents, user preferences, previous decisions

Use a database for facts that must be exact (status, IDs). Use a vector store for semantic search over text (documents, past tickets). Remember to manage size, relevance, privacy and stale data.

7. Human-in-the-Loop

A person reviews or approves the agent's output or actions before they take effect.

When to use it:

- High-risk actions (production changes, deleting data, running DML).
- Low model confidence or failed validation.
- Compliance or audit requirements.

Example: the agent produces a release risk assessment and a recommendation, but a release manager approves the final decision.

8. Guardrails

Guardrails are controls that keep the agent safe and within scope.

Area	Examples
Input	Validate and sanitize input, block prompt injection, limit size
Output	JSON schema validation, content checks, no sensitive data in output
Tools and permissions	Allow-list of tools, read-only access by default, least-privilege credentials
Actions	Human approval for risky actions, dry runs, rate limits
Limits	Max steps, timeout, max tokens, cost budget
Monitoring	Logging, audit trail, alerts

Principle: enforce rules in code and permissions, not only in the prompt.

9. Agent Evaluation

How do you know the agent works?

- Build a test set of realistic inputs with expected outputs (a "golden set").
- Measure task success rate, tool-calling accuracy (right tool, right parameters), output format validity, hallucination rate, latency and cost.
- For SQL: compare execution results with known correct results.
- Review logs and traces of the steps the agent took.
- Re-run the tests whenever the prompt, model or tools change (regression testing).
- Use human review for samples, and LLM-as-judge only as a supplement.
- Track production feedback and failures.

10. RAG Fundamentals

RAG (Retrieval-Augmented Generation) gives the LLM relevant external information at query time, so it answers from your data instead of only from memory.

```
Documents -> split into chunks -> embeddings -> vector store
Question -> embedding -> search similar chunks -> add to prompt -> LLM answers using them
```

Steps:

1. Chunk the documents (for example 300 to 800 tokens with some overlap).
2. Create embeddings (numeric vectors that capture meaning) and store them in a vector database.
3. At query time, embed the question and retrieve the most similar chunks.
4. Put the chunks in the prompt and ask the model to answer only from them, ideally citing the source.

Benefits: reduces hallucinations, uses current and private data, no retraining needed.

Common problems: poor chunking, irrelevant retrieval, stale data, and too much context. RAG is for knowledge in documents. For exact data in tables, use tools or SQL instead.

C. Your Project Architecture

How to answer any "how does your project work" question: use this structure.

1. Problem: what manual task or pain did it solve?
2. Flow: the steps from input to output.
3. Tools and APIs used.
4. Where the LLM is used, and where plain code is used.
5. Validation and safety.
6. Result: time saved, errors reduced (add real numbers if you have them).

Strong point to make: "The LLM never makes the final decision or runs anything unchecked. Deterministic code validates inputs and outputs."

1. How does your CRQ validation application work?

Template answer:

"The application automates validation of a change request (CRQ) before a release. It reads the CRQ details [from ITSM tool / API / form], then collects evidence from the source systems: code changes from GitHub and deployment/build status from Bamboo. A Python service runs the validation rules, for example [the CRQ lists the right components, the PR is approved and merged, the build and tests passed, the deployment matches the CRQ]. It calculates a risk score and produces a report. The LLM is used for [summarizing findings / checking the CRQ description against the code changes], and the output is validated JSON. Failed checks are flagged for a human reviewer."

```
CRQ input -> fetch GitHub data -> fetch Bamboo data -> rule checks -> risk score -> LLM summary -> report
```

Fill in: CRQ source, exact validation rules, what the output is, who reviews it.

2. How does Python communicate with GitHub?

Template answer:

"Python calls the GitHub REST API (using `requests` or PyGithub) over HTTPS. Authentication uses [a personal access token / GitHub App / enterprise token] stored in [a secrets manager or environment variable], never in code. I use endpoints such as [pull requests, commits, changed files, reviews, and status checks] to get the PR details for the CRQ. I handle pagination, rate limits and errors with retries and backoff, and log each call."

Example:

```
import requests

headers = {"Authorization": f"Bearer {token}", "Accept": "application/vnd.github+json"}
url = f"{base_url}/repos/{owner}/{repo}/pulls/{pr_number}/files"
resp = requests.get(url, headers=headers, timeout=30)
resp.raise_for_status()
files = resp.json()
```

Points: use least-privilege (read-only) tokens, set timeouts, handle pagination (`per_page`, `page`), respect rate limits, and for GitHub Enterprise use your enterprise base URL.

3. How does it validate Bamboo deployments?

Template answer:

"Python calls the Bamboo REST API with [an API token / service account] to get the build result and the deployment result for the release. I verify that [the build succeeded, tests passed, the deployed version or build number matches the version in the CRQ, it was deployed to the correct environment, and the deployment time is within the change window]. Any mismatch is marked as a validation failure with the reason."

Checks to mention (use the ones you built):

- Build status is successful.
- Deployed version equals the approved release version.
- Correct environment (for example pre-prod before prod).
- Deployment sequence and timing.
- No failed or rolled-back deployments for the release.

4. How do you calculate release risk?

Template answer:

"Release risk is calculated with a deterministic, explainable scoring model rather than by asking the LLM for a number. Each factor gets a weight, and the total maps to Low, Medium or High."

Example factors (use your real ones):

Factor	Example rule	Points
Size of change	Many files or lines changed	0 to 3
Critical components touched	Core pipelines, shared objects, production tables	0 to 4
Test or build status	Failed or missing tests	0 to 4
Past failures	Previous incidents on these components	0 to 3
Timing	Release at peak time or outside the window	0 to 2
Review	Missing approvals	0 to 3

```
0-3 = LOW    4-7 = MEDIUM    8+ = HIGH
```

"The LLM can then explain the score in plain language, but the score itself is calculated in code so it is consistent, auditable and testable."

5. How does your IICS automation authenticate?

Template answer:

"The automation uses the Informatica Cloud REST API. It logs in with a [service account username and password / API credentials], retrieves a session ID, and sends that session ID in the headers of later API calls. Credentials are stored in [a secrets manager / environment variables], not in code. The session is reused until it expires, then the script logs in again, and it logs out when done."

Points:

- The login returns a session ID and the base URL for subsequent calls.
- IICS has different API versions (v2 and v3), and they use different session header names, so confirm the one you used.
- Use a dedicated service account with least privilege, and mask secrets in logs.
- Handle session expiry and API failures with re-login and retries.
- Typical actions: start a Mapping Task or Taskflow, check job status, fetch logs.

6. How does your BigQuery agent validate data?

Template answer:

"The agent receives a validation request, for example 'compare source and target counts for table X', and uses the table schema as context to generate a SQL query. The SQL is checked by code (it must be a read-only SELECT on allowed tables, and pass a dry run). It is then executed, and the results are compared with the expected rules. Typical checks are [row counts, null checks, duplicate keys, sums, min/max dates, source-to-target reconciliation]. The agent returns a structured pass/fail result with the evidence, and failures go to a human."

```
Request -> get schema -> LLM generates SQL -> validate SQL (read-only, dry run) -> run -> evaluate result
```

7. How do you prevent incorrect AI-generated SQL?

Layered controls (say several of these):

1. Grounding: put real table and column names (schema) in the prompt, and say "use only these tables and columns."
2. Few-shot examples of correct SQL for common patterns.
3. Low temperature.
4. Static checks: parse the SQL and allow only SELECT. Block DML and DDL (INSERT, UPDATE, DELETE, DROP, MERGE). Allow-list datasets and tables.
5. BigQuery dry run: catches syntax errors, missing tables or columns, and shows bytes processed.
6. Cost guardrails: `maximum_bytes_billed`, partition filter required, and a row limit.
7. Read-only service account (BigQuery Data Viewer plus Job User), so even bad SQL cannot change data.
8. Result sanity checks: compare against a known query or a golden test set.
9. Human approval for anything complex or risky.
10. Log the prompt, SQL and result for audit.

Interview answer:

"I never execute generated SQL blindly. It goes through a schema-grounded prompt, a parser that allows only SELECT on approved tables, a BigQuery dry run for syntax and cost, and then runs with a read-only service account and a bytes-billed limit. The results are checked against expected rules, and risky cases go to a human."

8. How do you validate generated test data?

Generated (synthetic) test data must be checked by code, not trusted.

Checks:

- Schema: correct columns, data types and formats.
- Mandatory fields not null.
- Primary key uniqueness.
- Referential integrity (foreign keys exist in the parent table).
- Value ranges and allowed values (for example status is Active or Inactive, amount is positive).
- Business rules (for example end date is after start date).
- Volume: expected number of rows and a good spread of cases (normal, boundary, null and invalid cases).
- No real sensitive or personal data (masking or synthetic only).
- Reproducibility: use a seed and version the data.

Implementation: validate using Pydantic or a schema validator before loading, then run SQL checks (counts, duplicates) after loading.

9. How do you handle LLM failures?

Types of failure: API errors, timeouts, rate limits, invalid or malformed output, wrong or low-quality answers, and cost or latency spikes.

Handling:

Failure	Handling
Timeout or API error	Retry with exponential backoff, with a limit on retries
Rate limit	Backoff, queue requests, reduce concurrency
Invalid JSON	Validate, retry once with the error message, then fail safely
Wrong or unsafe output	Validation rules, human review, reject and log
Model unavailable	Fallback model or a rule-based path
Repeated failure	Circuit breaker, alert the team, mark the job as failed for manual handling
Long-running agent	Max steps, timeouts, token and cost budget

Also: make steps idempotent so retries do not duplicate actions, save state so a run can resume, log prompts and responses (without secrets) for debugging, and monitor failure rates.

Interview answer:

"I treat the LLM as an unreliable external service. Calls have timeouts and limited retries with backoff, outputs are validated against a schema, and there is a fallback path or human review when validation keeps failing. Critical decisions are made by deterministic code, so an LLM failure degrades the workflow but doesn't break or corrupt it."

D. Quick Revision

Topic	Remember
LLM	Predicts the next token, can hallucinate
Token and context window	Text unit; the context window is the input + output budget
Temperature	Low for SQL, JSON and automation
System vs user prompt	Rules and role vs the task
Structured output	Schema, then validate, retry once, then fail safely
Hallucination fix	Grounding, validation, low temperature, human review
Agent	LLM + tools + loop
Tool calling	Model proposes, your code validates and executes
Memory	Short-term (context) and long-term (database or vector store)
Human-in-the-loop	Approval for risky actions
Guardrails	Enforce in code and permissions, not only in the prompt
Evaluation	Golden test set, tool accuracy, regression tests
RAG	Retrieve relevant chunks, then generate from them
Core principle	Deterministic code for decisions, LLM for language and flexibility

30-second project pitch (adapt it):

"I built [project] to automate [manual process]. It uses Python to pull data from [GitHub, Bamboo, IICS, BigQuery] through their APIs, applies deterministic validation and a risk scoring model, and uses an LLM only to [summarize or generate SQL]. LLM output is schema-validated, SQL is checked and run read-only, and risky cases go to a human. This reduced [manual effort or errors] by [real number]."

Be ready for these follow-ups: "What would you improve?", "How do you test it?", "What was the hardest problem?", "How do you secure the credentials?"